

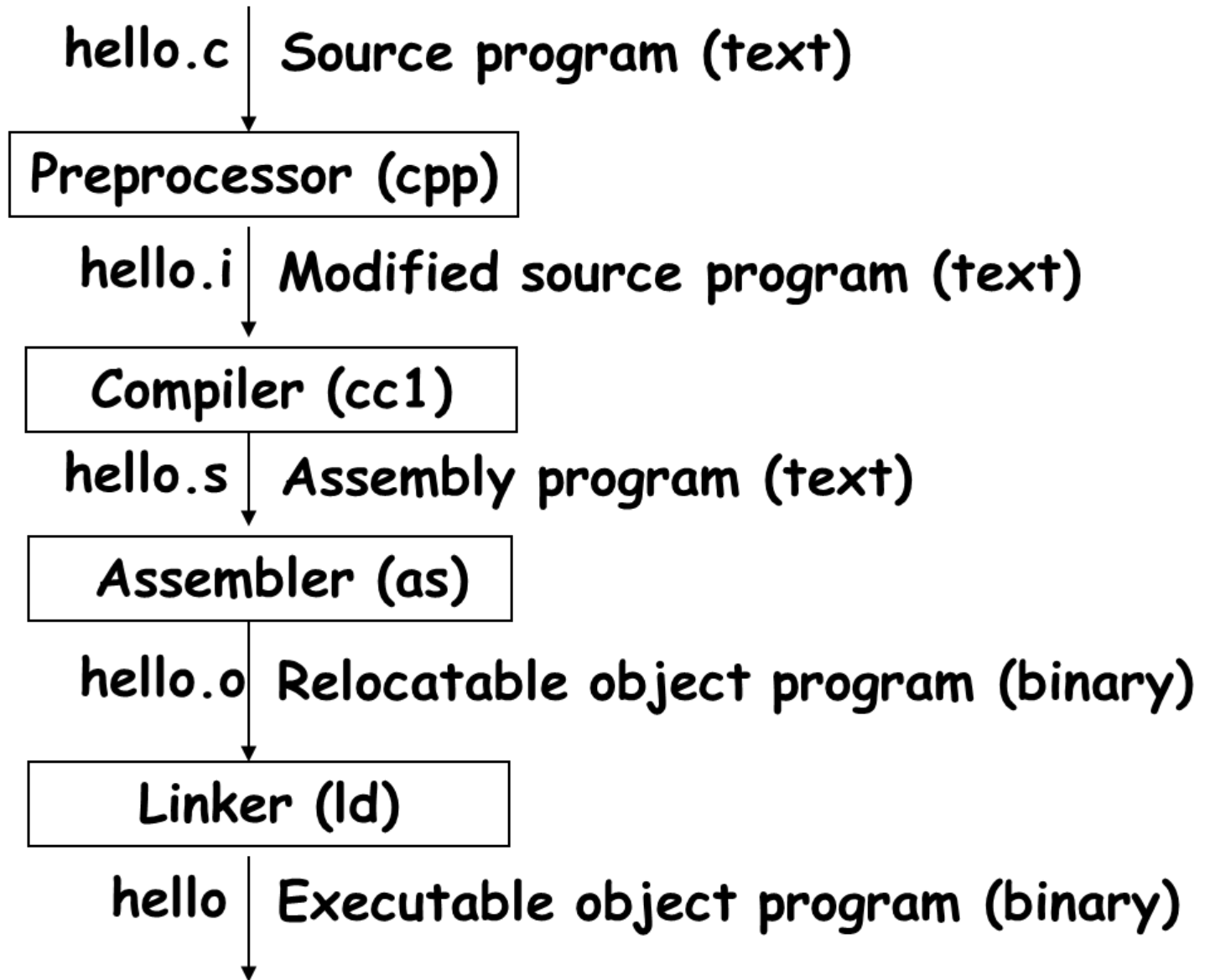
compiler

```
gcc -E test.c > test-pp.c (test-pp.c)
gcc -S test.c (test.s)
gcc -c test.c (test.o)
gcc -o test test.c (test)
```

make

```
1 CC := gcc
2 OPT := -O3
3 CFLAGS := -Wall -I ./include $(OPT)
4
5 BINS := hello-1 hello-2 hello-3
6
7 all: $(BINS)
8
9 hello-1.o: hello-1.c
10      gcc -c hello-1.c -Wall -O3
11
12 hello-2.o: hello-2.c
13      $(CC) -c hello-2.c $(CFLAGS)
14
15 hello-3.o: hello-3.c
16      $(CC) -c $^ $(CFLAGS)
17
18 clean:
19      rm -f $(BINS) *.o
```

bit



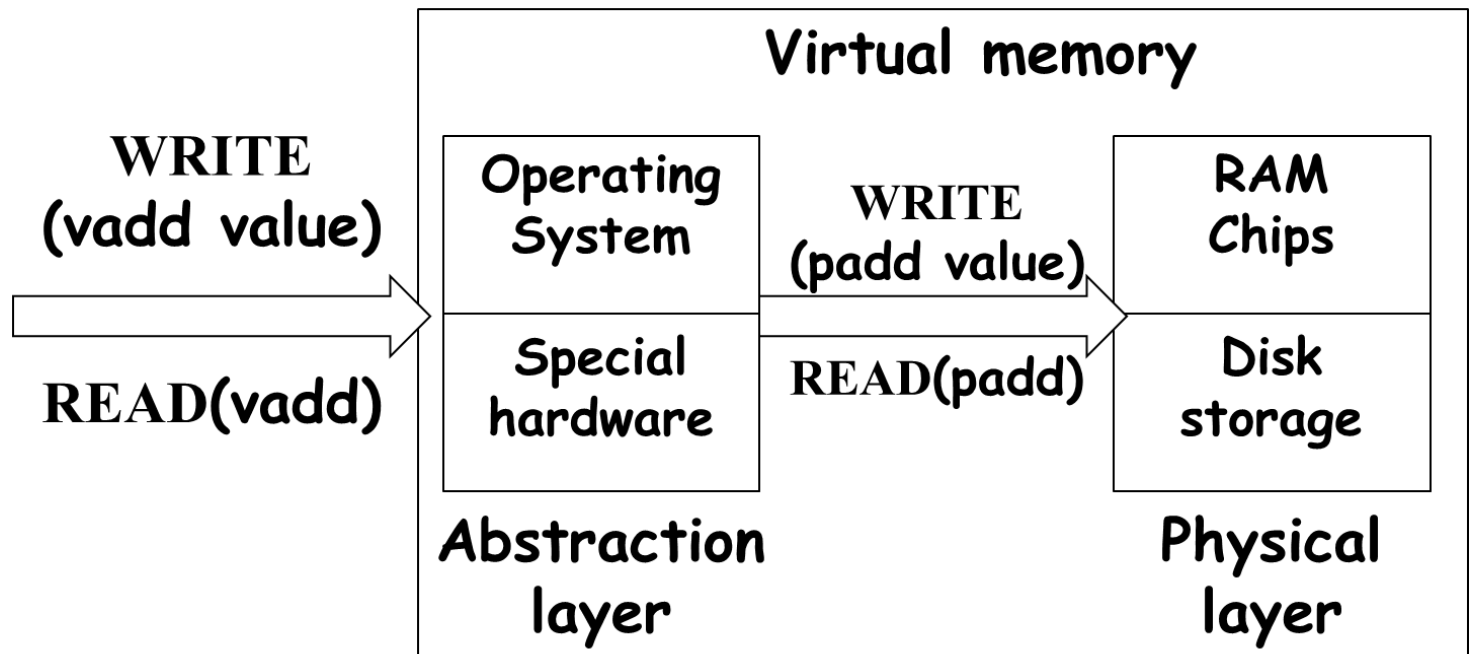
storage

data size

C Declaration		Bytes	
Signed	Unsigned	32-bit	64-bit
char	unsigned char	1	1
short	unsigned short	2	2
int	unsigned	4	4
long	unsigned long	4	8
int32_t	uint_32	4	4
int64_t	uint_64	8	8
char *		4	8
float		4	4
double		8	8

virtual memory

- Hardware
 - random-access memory (RAM) (physical)
 - disk storage (physical)
 - special hardware (performing the abstraction)
- Software
 - and operating system software (abstraction)



– 80483bd: 01 05 64 94 04 08->add %eax, 0x8049464

byte ordering

Big Endian (0x1234567)

0x100 0x101 0x102 0x103

		01	23	45	67		
--	--	----	----	----	----	--	--

Little Endian (0x1234567)

0x100 0x101 0x102 0x103

		67	45	23	01		
--	--	----	----	----	----	--	--

encoding

- Bit vector $[x_{w-1}, x_{w-2}, x_{w-3}, \dots, x_0]$

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

twos-complement

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

正数的范围是0到 $2^{(w-1)}-1$ ，而负数的范围是-1到 $-2^{(w-1)}$ 。

unsigned和signed比较，统一转成unsigned

$$T2U(-2147483647) = 2^{32}-2147483647 = 2147483648$$

Constant1	Constant2	Relation	Evaluation
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483648	>	signed
2147483647U	-2147483648	<	unsigned
-1	-2	>	signed
(unsigned)-1	-2	>	unsigned

size_t是unsigned

```
int strlonger(char *s, char *t) {
    return strlen(s) - strlen(t) > 0;
}
```

operation

multiplication

两个w位的bit-vector相乘的结果的前w位相同

- **Unsigned**
 - $x *_w^u y = (x * y) \bmod 2^w$
 - **Signed**
 - $x *_w^t y = U2T((x * y) \bmod 2^w)$
 - **Given two bit vectors $\vec{x}$ and $\vec{y}$**
 - $x = B2T_w(\vec{x}), y = B2T_w(\vec{y}), x' = B2U_w(\vec{x}), y' = B2U_w(\vec{y})$
 - $T2B_w(x *_w^t y) = U2B_w(x' *_w^u y')$
 - $x' = x + x_{w-1}2^w, y' = y + y_{w-1}2^w \quad (x' * y') \bmod 2^w = (x * y) \bmod 2^w$
 - $T2U_w(x *_w^t y) = T2U_w(U2T((x * y) \bmod 2^w)) = (x * y) \bmod 2^w = x' *_w^u y'$
- $$T2B_w(x *_w^t y) = UTB_w(T2U_w(x *_w^t y)) = UTB_w(x' *_w^t y')$$

Mode	x		y		$x \cdot y$		Truncated $x \cdot y$	
Unsigned	5	[101]	3	[011]	15	[001111]	7	[111]
Two's complement	-3	[101]	3	[011]	-9	[110111]	-1	[111]
Unsigned	4	[100]	7	[111]	28	[011100]	4	[100]
Two's complement	-4	[100]	-1	[111]	4	[000100]	-4	[100]
Unsigned	3	[011]	3	[011]	9	[001001]	1	[001]
Two's complement	3	[011]	3	[011]	9	[001001]	1	[001]

4-bit	(x + y)	(x - y)	(x * y)	(-y)
(x = 4, y = 7)	-5	-3	-4	-7
(x = -6, y = -8)	2	2	0	-8
(x = 5, y = -1)	4	6	-5	1
(x = -3, y = 6)	3	7	-2	-6

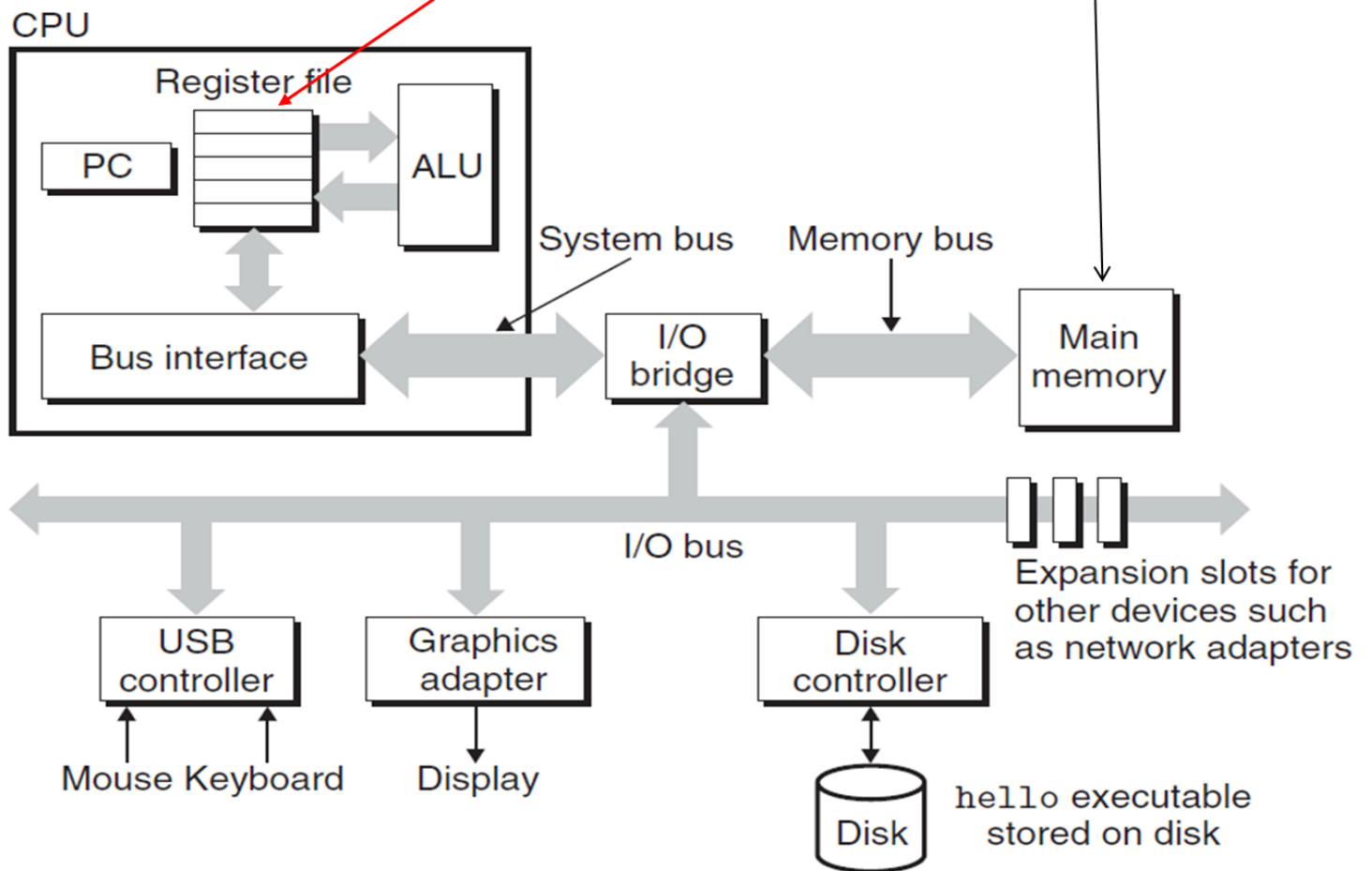
-3 * 6解法

1. $-3 * 6 = U2T(13 * 6 \bmod 16) = U2T(14) = 14 - 16 = -2$
2. $-3 * 6 = -18 = \sim(0b10010) + 1 = 0b01110$
 $\text{trunc}(0b01110) = 0b1110 = -2$

Machine-Level Representation of Programs

Register

Memory



registrer

16 8-byte integer registers

63	31	15	7	0				
%rax					%eax	%ax	%al	Return value
%rbx					%ebx	%bx	%bl	Callee saved
%rcx					%ecx	%cx	%cl	4th argument
%rdx					%edx	%dx	%dl	3rd argument
%rsi					%esi	%si	%sil	2nd argument
%rdi					%edi	%di	%dil	1st argument
%rbp					%ebp	%bp	%bpl	Callee saved
%rsp					%esp	%sp	%spl	Stack pointer
%r8					%r8d	%r8w	%r8b	5th argument
%r9					%r9d	%r9w	%r9b	6th argument
%r10					%r10d	%r10w	%r10b	Caller saved
%r11					%r11d	%r11w	%r11b	Caller saved
%r12					%r12d	%r12w	%r12b	Callee saved
%r13					%r13d	%r13w	%r13b	Callee saved
%r14					%r14d	%r14w	%r14b	Callee saved
%r15					%r15d	%r15w	%r15b	Callee saved

16 floating point registers

255	127	0	
%ymm0	%xmm0		1st FP arg./Return value
%ymm1	%xmm1		2nd FP argument
%ymm2	%xmm2		3rd FP argument
%ymm3	%xmm3		4th FP argument
%ymm4	%xmm4		5th FP argument
%ymm5	%xmm5		6th FP argument
%ymm6	%xmm6		7th FP argument
%ymm7	%xmm7		8th FP argument

%ymm8	%xmm8	Caller saved
%ymm9	%xmm9	Caller saved
%ymm10	%xmm10	Caller saved
%ymm11	%xmm11	Caller saved
%ymm12	%xmm12	Caller saved
%ymm13	%ymm13	Caller saved
%ymm14	%xmm14	Caller saved
%ymm15	%xmm15	Caller saved

address

- Imm(r_b, r_i, s)

```
function(%rdi, %rsi, %rdx)
    return %rax
```

Data movement

- (immediate, register)
- (memory, register) load
- (register, register)
- (immediate, memory) limited
- (register, memory) store

- w 字(2 bytes) 16位
- d, l 双字 32位
- q 四字 64位
 - mov (general)
 - movb (move byte)
 - movw (move word)
 - movl (move double word)
 - movq (move quad word)
 - Immediate can only be represented as 32-bit signed sign extension to the destination
 - movabsq (move absolute quad word)
- Only update the specific register byte
 - except movl which updates the higher-order 4 bytes of the register to 0

unsigned

- movz (move with zero extension)
- movzbw (move zero-extension byte to word)
- movzbl (move zero-extension byte to double word)
- movzbq (move zero-extension byte to quad word)
- movzwl (move zero-extension word to double word)
- movzwq (move zero-extension word to quad word)
- no movzld, it is implemented by movl

signed

- movs (move with sign extension)
- movsbw (move sign-extension byte to word)
- movsbl (move sign-extension byte to double word)
- movsbq (move sign-extension byte to quad word)
- movswl (move sign-extension word to double word)
- movswq (move sign-extension word to quad word)
- movslq (move sign-extension double word to quad word)

example

Initial value %dl=8d %rax=0000000098765432

- 1 movb %dl, %al %rax=000000009876548d
- 2 movsbl %dl, %eax %rax=00000000ffffff8d
- 3 movzbq %dl, %rax %rax=000000000000008d

Stack

%rsp register stack pointer

```

int main()
{
    int a = 4 , b = 3, c = 2;
    decode1(&a, &b, &c);
    printf(“a = %d, b = %d, c = %d\n”, a, b, c);
}

```

Data manipulation

注意

- sub S, D 是 $D \leftarrow D - S$ 被减数和存入的都在右侧
- sar算术右移，shr逻辑右移
- idivq S右边有长整数除法

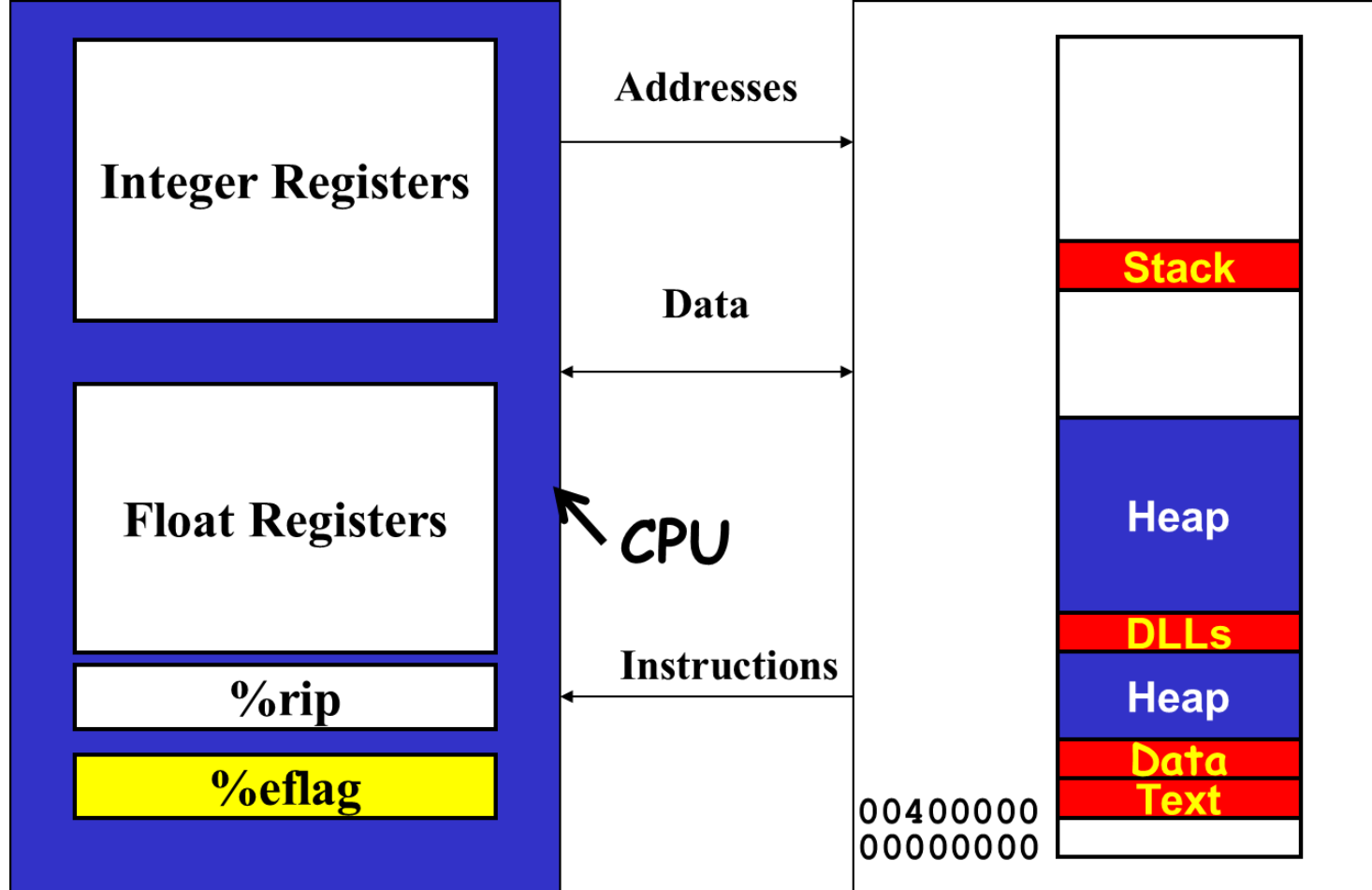
Instruction	Effect	Description
leaq S, D	$D \leftarrow \&S$	Load effective address
inc D	$D \leftarrow D + 1$	Increment
dec D	$D \leftarrow D - 1$	Decrement
neg D	$D \leftarrow -D$	Negate
not D	$D \leftarrow \sim D$	Complement
add S, D	$D \leftarrow D + S$	Add
sub S, D	$D \leftarrow D - S$	Subtract
imul S, D	$D \leftarrow D * S$	Multiply

Instruction	Effect	Description
xor S, D	$D \leftarrow D \wedge S$	Exclusive-or
or S, D	$D \leftarrow D \mid S$	Or
and S, D	$D \leftarrow D \& S$	And
sal k, D	$D \leftarrow D \ll k$	Left shift
shl k, D	$D \leftarrow D \ll k$	Left shift
sar k, D	$D \leftarrow D \gg k$	Arithmetic right shift
shr k, D	$D \leftarrow D \gg k$	Logical right shift

imulq S	$R[\%rdx]:R[\%rax] \leftarrow S * R[\%rax]$	Signed full multiply
mulq S	$R[\%rdx]:R[\%rax] \leftarrow S * R[\%rax]$	Unsigned full multiply
cqto	$R[\%rdx]:R[\%rax] \leftarrow \text{SignExtend}(R[\%rax])$	Convert to oct word
idivq S	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Signed divide
divq S	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Unsigned divide

control

%rip PC 64位
%eflag 32位



change CD

%eflag 存condition codes 32位

- CF: carry flag
- OF: overflow flag
- ZF: zero flag
- SF: sign flag

- lea instruction
 - has no effect on condition codes
- Xorl instruction
 - The carry and overflow flags are set to 0
- Shift instruction
 - carry flag is set to the last bit shifted out
 - Overflow flag is set to 0

- **test b,a** like computing $a \& b$ without setting destination
- ZF set when $a \& b == 0$
- SF set when $a \& b < 0$

指令	基于	描述
CMP S_1, S_2	$S_2 - S_1$	比较
cmpb		比较字节
cmpw		比较字
cmpl		比较双字
cmpq		比较四字
TEST S_1, S_2	$S_1 \& S_2$	测试
testb		测试字节
testw		测试字
testl		测试双字
testq		测试四字

- **cmp b,a** like computing $a - b$ without setting destination
- CF set if carry out from most significant bit
 - Used for unsigned comparisons
- ZF set if $a == b$
- SF set if $(a - b) < 0$
- OF set if two's complement overflow
 - $(a > 0 \ \&\& \ b < 0 \ \&\& \ (a - b) < 0) \ ||$
 - $(a < 0 \ \&\& \ b > 0 \ \&\& \ (a - b) > 0)$

read CD

Instruction	Synonym	Effect	Set Condition
Sete	Setz	ZF	Equal/zero
Setne	Setnz	~ZF	Not equal/not zero
Sets		SF	Negative
Setns		~SF	Nonnegative
Setl	Setnge	SF^OF	Less
Setle	Setng	(SF^OF) ZF	Less or Equal
Setg	Setnle	~(SF^OF)&~ZF	Greater
Setge	Setnl	~(SF^OF)	Greater or Equal
Seta	Setnbe	~CF&~ZF	Above
Setae	Setnb	~CF	Above or equal
Setb	Setnae	CF	Below
Setbe	Setna	CF ZF	Below or equal

jmp

rep作为一个优化，不执行动作。

loop:

```
1 4004d0: 48 89 f8 movq %rdi, %rax
2 4004d3: eb 03      jmp 8<loop+0x8>
3 4004d5: 48 di f8 sari %rax
4 4004d8: 48 85 c0    testq %rax, %rax
5 4004db: 7f f8      jq 5<loop+0x5>
6 4004dd: f3 c3      repz retq
```

03 + 4004d5 = **4004d8**

f8(-8) + 4004dd = **4004d5**

如果有lt_cnt和ge_cnt

```

long lt_cnt = 0 ;
long ge_cnt = 0 ;
long absdiff_se(long x, long y)
{
    long result ;
    if (x < y) {
        lt_cnt++
        return y - x;
    }
    else {
        ge_cnt++ ;
        return x - y;
    }
}

```

```

1. long gotodiff_se(long x, long y)
2. {
3.     long result ;
4.     if (x >= y) goto x_ge_y ;
5.     lt_cnt++ ;
6.     result = y - x ;
7.     return result ;
8.     x_ge_y:
9.     ge_cnt++ ;
10.    result = x - y;
11.    return result;
12. }

```

14

(a) Original C code

```

long absdiff(long x, long y)
{
    long result ;
    if (x < y)
        restul = y-x ;
    else
        result = x-y ;
    return result ;
}

```

(b) Implementation using conditional assignment

```

1 long cmovdiff(long x, long y) {
2     long rval = y-x;
3     long eval = x-y;
4     long ntest = x >= y;
5     /* Line below requires
6         single instruction: */
7     if (ntest) rval = eval;
8     return rval;
9 }

```

16

conditional move

源寄存器或内存地址S，目的寄存器R

Instruction		Synonym	Move condition	Description
<code>cmove</code>	S, R	<code>cmovz</code>	ZF	Equal / zero
<code>cmovne</code>	S, R	<code>cmovnz</code>	$\sim$ ZF	Not equal / not zero
<code>cmovs</code>	S, R		SF	Negative
<code>cmovns</code>	S, R		$\sim$ SF	Nonnegative
<code>cmovg</code>	S, R	<code>cmovnle</code>	$\sim(SF \wedge OF) \ \& \ \sim ZF$	Greater (signed >)
<code>cmovge</code>	S, R	<code>cmovnl</code>	$\sim(SF \wedge OF)$	Greater or equal (signed >=)
<code>cmovl</code>	S, R	<code>cmovnge</code>	$SF \wedge OF$	Less (signed <)
<code>cmovle</code>	S, R	<code>cmovng</code>	$(SF \wedge OF) \mid ZF$	Less or equal (signed <=)
<code>cmova</code>	S, R	<code>cmovnbe</code>	$\sim CF \ \& \ \sim ZF$	Above (unsigned >)
<code>cmovae</code>	S, R	<code>cmovnb</code>	$\sim CF$	Above or equal (Unsigned >=)
<code>cmovb</code>	S, R	<code>cmovnae</code>	CF	Below (unsigned <)
<code>cmovbe</code>	S, R	<code>cmovna</code>	$CF \mid ZF$	below or equal (unsigned <=)

destination must be register. source doesn't matter

不能用cmove, *xp始终会执行

```
int cread(int *xp) {
    return (xp ? *xp : 0);
}
```

when are the use of registers not saved

- Leave procedures
- Inter-procedure register allocation
- a1 is dead before f calls h()
- Register window

jmp table

```

int switch_eg_impl (long x,
                    long n, long *dest)
{
    static void *jt[7] = { &&loc_A,
                          &&loc_def, &&loc_B,
                          &&loc_C, &&loc_D,
                          &&loc_def, &&loc_D
                        };

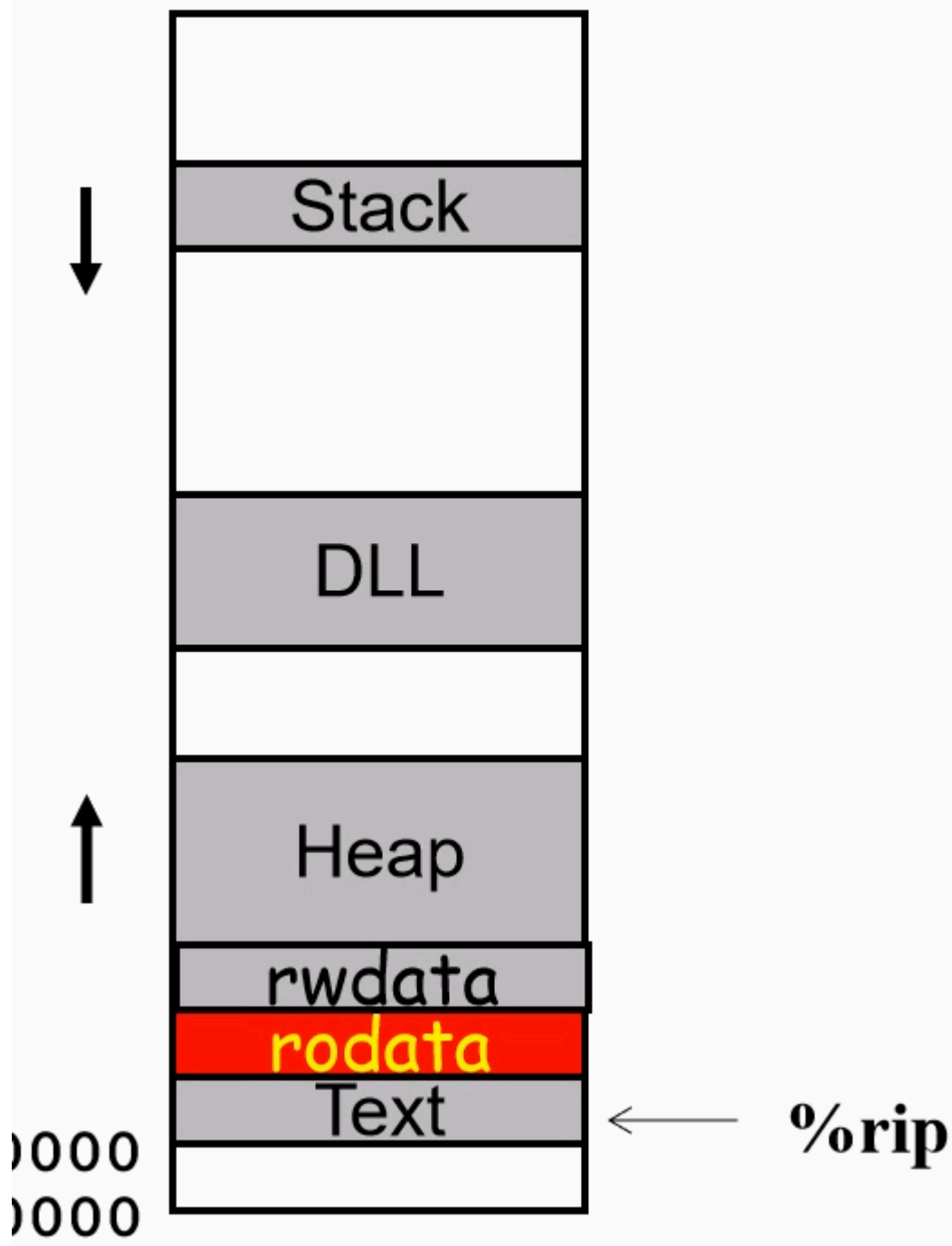
    unsigned long index = n - 100;
    long val ;
    if ( index > 6 )
        goto loc_def ; //default
    goto *jt[index];    // multiway
loc_A: // case 100
    val = x * 13 ;
    goto done ;

```

```

loc_B: // case 102
    x = x + 10 ; // fall through
loc_C: // case 103
    val = x + 11;
    goto done ;
loc_D: // case 104, 106
    val *= x * x ;
    goto done ;
loc_def: //default
    val = 0 ;
done:
    *dest = val ;
}

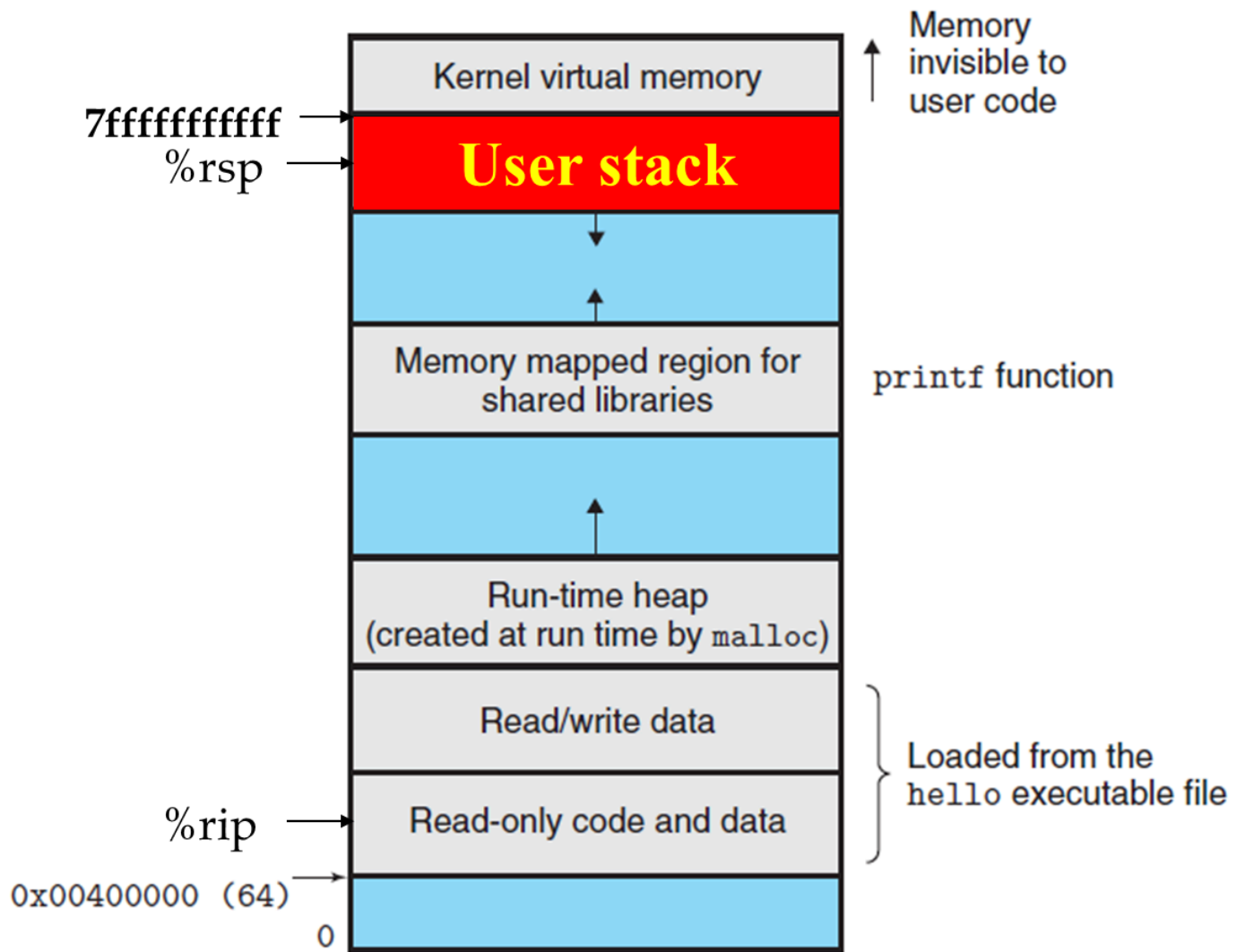
```



procedure call

jmp with: return, pass data, local variable, register

caller: 调用者 callee: 被调用者



- `call` = `push` + `jmp`
- `ret` = `pop` + `jmp`

example

// beginning of function **multstore**

1. 400540 <multstore>:

2. **400540**: 53

push %rbx

3. 400541: 48 89 d3

mov %rdx, %rbx

...

// return from function **multstore**

4. 40054d: c3

retq

...

//call to **mulstore** from **main**

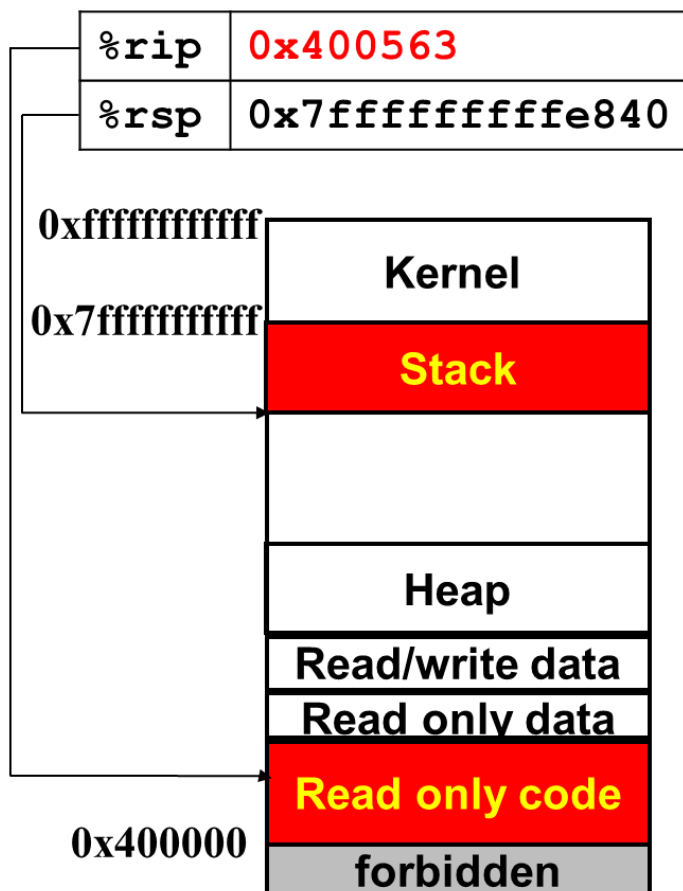
5. **400563**: e8 d8 ff ff ff

callq 400540<multstore>

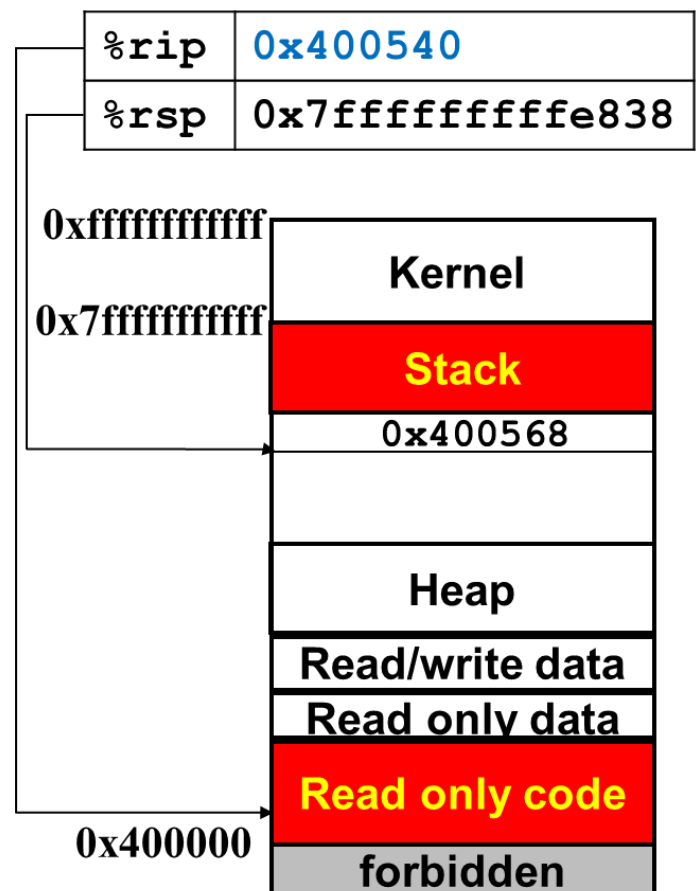
6. **400568**: 48 8b 54 24 08

mov 0x8(%rsp), %rdx

Executing call



After call



Operand size	Argument number					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

多于6个参数

- Pushed by Caller to stack
 - Saved in caller frame
 - Just upon of return address
 - From **Nth** to **7th** (from right to left)
 - All data sizes are rounded up to be multiples of eight
- How to be used by Callee
 - Relative to **%rsp**

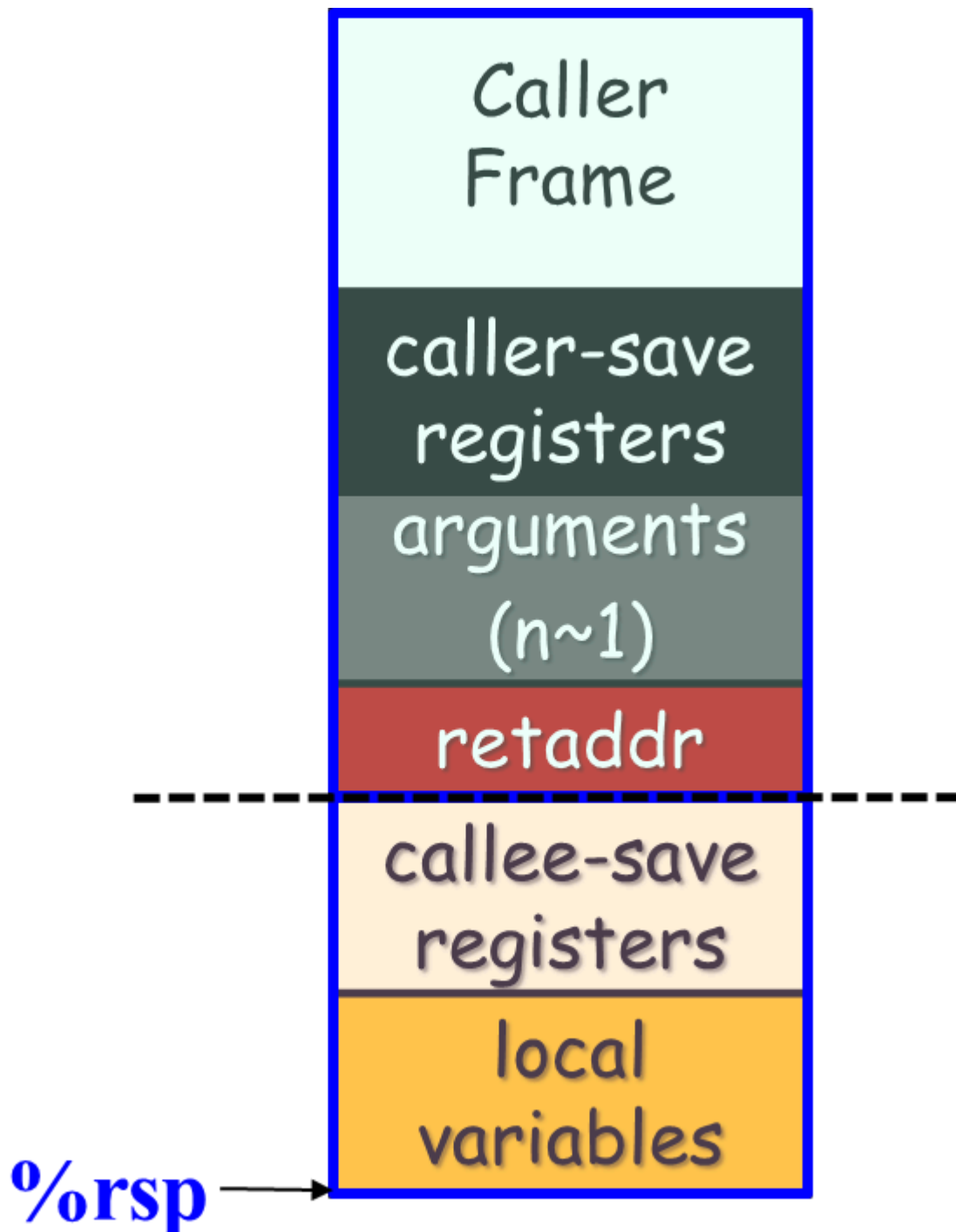
caller-save callee可以随便修改, caller保存才能恢复

- Caller-save registers
 - **%rax, %rdi, %rsi, %rdx, %rcx, %r8, %r9, %r10, %r11**

•

callee-save 对于caller来说, callee不会修改这些值

- Callee-save registers
 - %rbx, %rbp, %r12-15



```

long P(long x, long y)
{
    long u = Q(y);
    long v = Q(x);
    return u + v;
}

```

1	P:		
2	pushq	%rbp	<i>Save %rbp</i>
3	pushq	%rbx	<i>Save %rbx</i>
4	subq	\$8, %rsp	<i>Align stack frame</i>
5	movq	%rdi, %rbp	<i>Save x</i>
6	movq	%rsi, %rdi	<i>Move y to first argument</i>
7	call Q	Call Q(y)	
8	movq	%rax, %rbx	<i>Save result</i>
9	movq	%rbp, %rdi	<i>Move x to first argument</i>
10	call Q	Call Q(x)	
11	addq	%rbx, %rax	<i>Add saved Q(y) to Q(x)</i>
12	addq	\$8, %rsp	<i>Deallocate last part of stack</i>
13	popq	%rbx	<i>Restore %rbx</i>
14	popq	%rbp	<i>Restore %rbp</i>
15	ret		

array

```
int var_ele(long n, int A[n][n], long i, long j)
{
    return A[i][j];
}
```

- Declare an array `int A[exp1][exp2]`
 - either as a local variable
 - or as an argument to a function

Heterogeneous Data Structures

struct & union

```
struct S3 {
    char c;
    int i[2];
    double v;
};
union U3 {
    char c;
    int i[2];
    double v;
};
```

Type	c	i	v	size
S3	0	4	16	24
U3	0	0	0	8

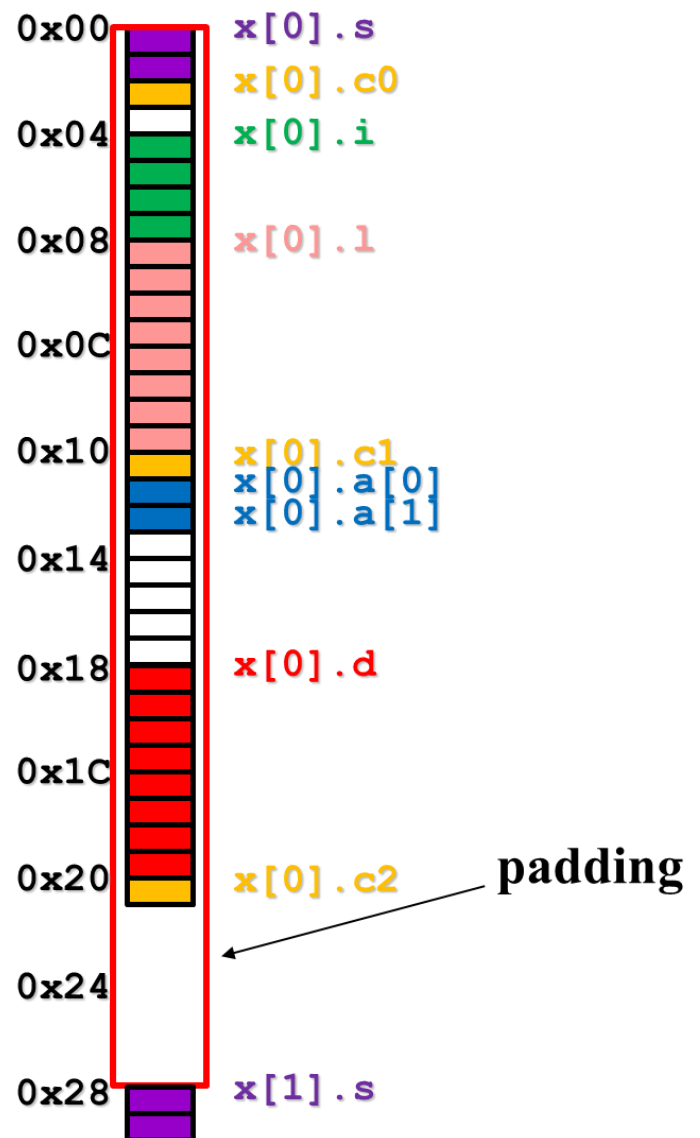
align

- Linux alignment restriction
 - 1-byte data types are able to have any address
 - 2-byte data types must have an address that is multiple of 2
 - 4-byte data types must have an address that is multiple of 4
 - Any larger data types must have an address that is multiple of 8

example

```
struct xxx {
    short s;
    char c0;
    int i;
    long l;
    char c1;
    char a[2];
    double d;
    char c2;
};

struct xxx x[2];
```

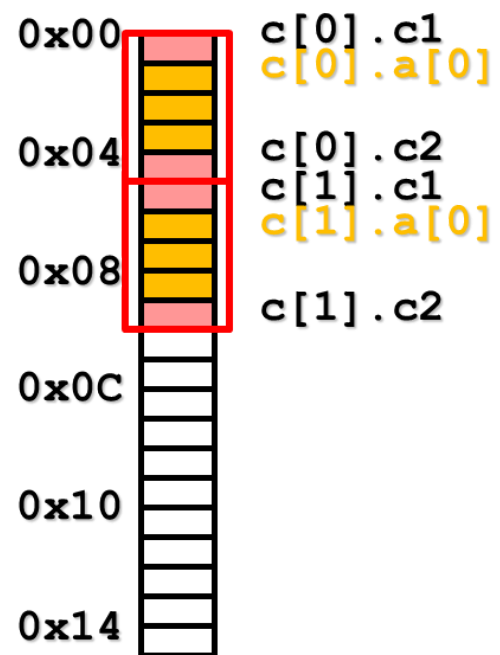


```

struct ccc {
    char c1;
    char a[3];
    char c2;
};

struct ccc c[2];

```



buffer overflow

prevent

1. stack randomization
2. stack corruption detect: canary

```

1 echo:
2     subq    $24, %rsp                # Allocate 24 bytes on stack
3     movq    %fs:40, %rax             # Retrieve canary
4     movq    %rax, 8(%rsp)            # Store on stack
5     xorl    %eax, %eax              # Zero out register
6     movq    %rsp, %rdi              # Compute buf as %rsp
7     call    gets                    # Call gets
8     movq    %rsp, %rdi              # Compute buf as %rsp
9     call    puts                    # Call puts
10    movq    8(%rsp), %rax            # Retrieve canary
11    xorq    %fs:40, %rax            # Compare to stored value
12    je      .L9                     # If =, goto ok
13    call    __stack_chk_fail        # Stack corrupted!
14 .L9                                # ok:
15    addq    $24, %rsp                # Deallocate stack space
16    ret

```

3. limit excutable code region: stack not excutable

return-oriented programming

pointer

- Addition and subtraction
 - $p+i$, $p-i$ (result is a pointer)
 - $p-q$ (result is a long)

```
int numcmp(char *s1, char *s2)
{
    double v1, v2;

    v1 = atof(s1);
    v2 = atof(s2);
    if (v1 < v2) return -1;
    else if ( v1 > v2 ) return 1;
    else return 0;
}
```

```
/* strcmp: compare s1 and s2 as string */
int strcmp(char *s1, char *s2);
```

```

#include <stdio.h>
#include <string.h>

{
    ...
    int numeric = 0, (*cmp) (void *, void *);
    char *s1, *s2;
    ...
    if (...) numeric = 1 ;
    ...
    cmp = (int (*)(void *, void *))
           (numeric ? numcmp : strcmp);
    (*cmp) (s1, s2);
    ...
}

```

- char (*(*x())[])()
 - Function returning pointer to array[] of pointer to function returning char
- char (*(*x[3])()) [5]
 - Array[3] of pointer to function returning pointer to array[5] of char

Operators	Associativity
() [] -> . ++ --	left to right
! ~ ++ -- + - * & (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
?:	right to left
= += -= *= /= %= &= ^= != <<= >>=	right to left
,	left to right

Note: Unary +, -, and * have higher precedence than binary forms

